MINDANAO STATE UNIVERSITY - ILIGAN INSTITUTE OF TECHNOLOGY
College of Computer Studies
Department of Computer Applications
BCA182 Embedded Systems Programming
LABORATORY ACTIVITY NO. 2

LABORATORY REPORT: PERSONAL MP3 PLAYER

Member Name	Sherie Grace L. Lupoz
Member Name	Renato G. Beroy
Member Name	Binyusoph P. Sansarona
Date	October 20, 2026
Subject/Section	BCA182 [B187-B187.1]
Development Platform	RT-Thread RT-Spark (STM32F407) + FreeRTOS + PlatformIO

1. Problem and Requirements

This laboratory activity focused on the design and implementation of a personal MP3 player on the RT-Thread RT-Spark development board, using PlatformIO, the STM32Cube HAL and FreeRTOS. The player plays songs through the board's audio codec, shows the song name and player state on the on-board LCD, shows the state on LEDs, reads a potentiometer for volume, and prints its instructions over UART.

The handout requires the following behavior:

- Buttons 2 to 4 enter a song number in binary, and button 1 selects the song. With all buttons released, the code is zero and the first song is chosen.
- Button 1 must be pressed again to confirm the choice. If nothing is done for 5 seconds, everything continues normally.
- The song name is displayed on the LCD while playing, and a confirmation message is displayed when the song is changed.
- A separate button (USER_BUTTON) stops and plays the song.
- LEDs show the player state: one color for playing, one for paused, one for changing the song.
- A potentiometer adjusts the volume.
- The instructions for using the player are sent over UART.
- The features run in three threads, each with an infinite loop, and a mutex protects the LCD.
- The main program clears the LCD, starts the threads, sends the instructions and uses sleep mode to reduce power consumption.
- The note playback (ticker) and the 5-second confirmation (timeout) use timer interfaces.

The required tools were PlatformIO, the STM32 framework (the Arduino framework is prohibited), FreeRTOS or Zephyr, and the RT-Spark board. The handout states that 8 songs can be played,

while the provided song_def.h defines 10 songs. Three code buttons give only 8 codes, so the first 8 songs were used.

2. System Architecture and Design

2.1 Hardware

- RT-Spark board: STM32F407ZGT6 (Cortex-M4F, 168 MHz, 8 MHz crystal).
- On-board 1.3 inch 240x240 LCD with an ST7789 driver on the FSMC 8-bit (8080) bus.
- On-board ES8388 stereo audio codec, controlled over I2C2 and fed audio over I2S3; the 3.5 mm headphone jack drives the speaker.
- On-board keys UP, LEFT, DOWN and RIGHT, used as buttons 1 to 4.
- On a breadboard: three LEDs with 220 ohm resistors, a 10 kilo-ohm potentiometer, and one push button (USER_BUTTON) connected between PA0 and GND.
- ST-Link USB connection for programming, power and a virtual COM port (USART1).

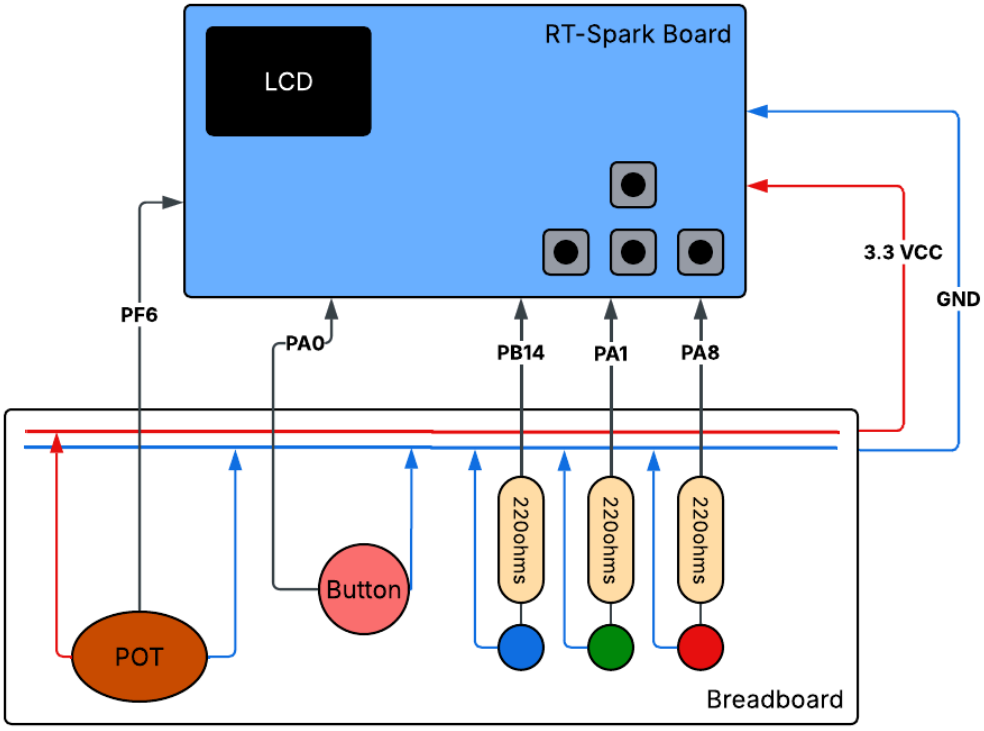


Figure 2.1 Block diagram of MP3 Player

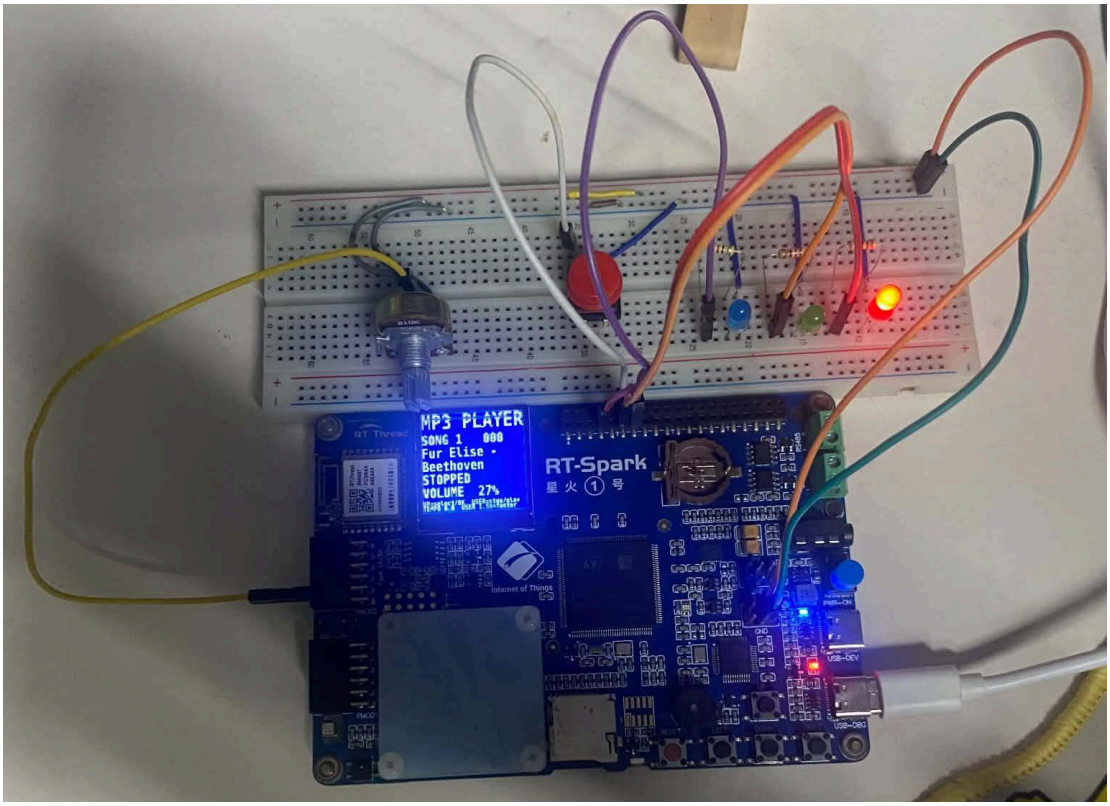


Figure 2.2 Actual wiring of MP3 Player

Pin Configuration

Component	STM32 Pin
Blue LED (playing)	PB14
Green LED (changing song)	PA1
Red LED (paused / stopped)	PA8
Potentiometer (ADC3 channel 4)	PF6
Button 1 / UP	PC5
Button 2 / LEFT	PC0
Button 3 / DOWN	PC1
Button 4 / RIGHT	PC4
USER_BUTTON (breadboard, to GND)	PA0
USART1 TX / RX	PA9 / PA10
LCD data D0-D7	PD14, PD15, PD0, PD1, PE7, PE8, PE9, PE10
LCD RS (A18), RD, WR, CS (NE3)	PD13, PD4, PD5, PG10

Component	STM32 Pin
LCD reset, backlight	PD3, PF9
Codec I2C2 SCL / SDA	PF1 / PF0
Codec I2S3 MCK, WS, CK, SD	PC7, PA15, PB3, PB5

2.2 Software Structure

The firmware is written in C with the STM32Cube HAL and native FreeRTOS APIs. The course song data is C++, so it is isolated in one file:

- src/main.c: hardware setup, threads, timers, audio engine and button logic.
- src/lcd.c, include/lcd.h, include/lcd_font.h: ST7789 driver and text drawing.
- src/songs.cpp, include/songs.h: a C wrapper around the instructor's song.h and song_def.h, which are used unchanged.
- include/FreeRTOSConfig.h, platformio.ini, fpu_link.py: project configuration.

2.3 Player State Machine

The user interface has two states. In IDLE the current song is playing, paused or stopped. A tap of button 1 with a song code moves the player to CONFIRM, which starts a 5-second window. A second tap of button 1 in CONFIRM confirms the song: the player stops, switches to the chosen song and plays it from the start, then returns to IDLE. If 5 seconds pass without a confirmation, the timeout returns the player to IDLE and the current song carries on. In IDLE, a tap of the USER_BUTTON pauses or resumes the song. When a song reaches its last note, it stops and can be replayed.

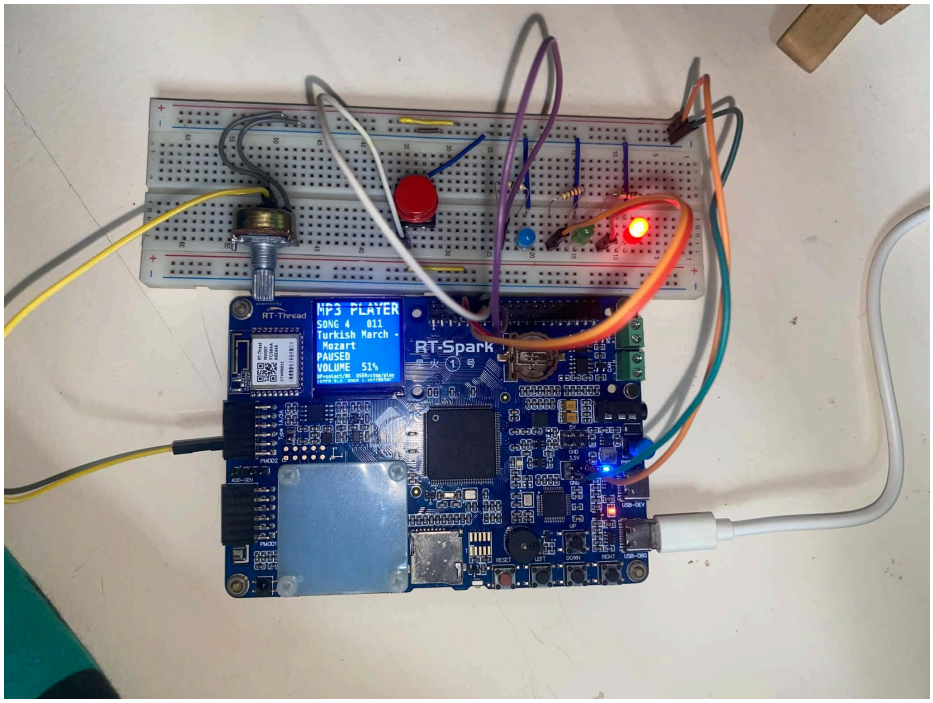


Figure 2.3 Player State Machine - STOPPED/PAUSED

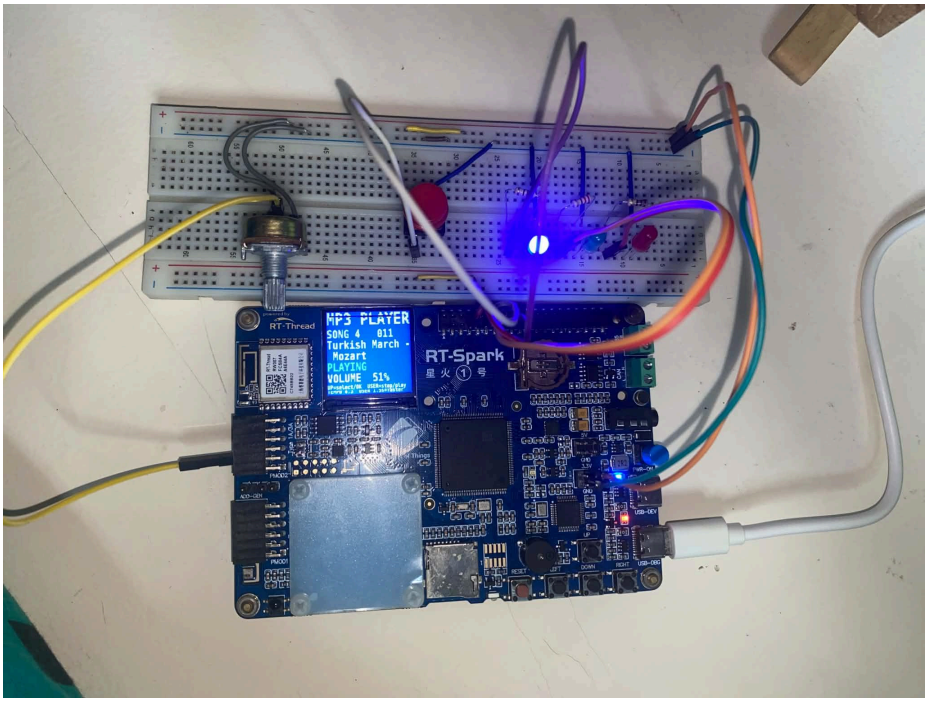


Figure 2.4 Player State Machine - PLAYING

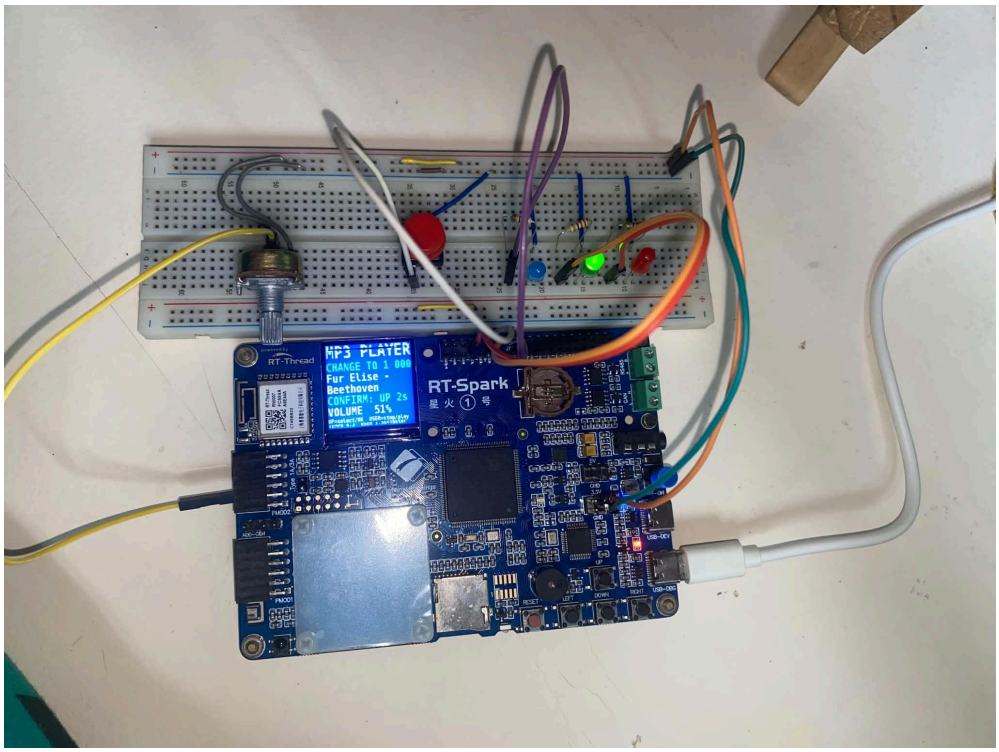


Figure 2.5 Player State Machine - CHANGING SONG

3. FreeRTOS Architecture

The application is split into three threads named as in the handout, plus a start-up task and two software timers.

Task / Timer	Responsibility	Period	Priority	Blocks on
update_lcd_leds_thread	Draws the LCD and drives the LEDs	50 ms	1	vTaskDelay, LCD mutex
polling_buttons	Reads and debounces the keys, runs the song choice logic	20 ms	1	vTaskDelay, UART mutex
adjust_volume	Reads the potentiometer, sets the codec volume, prints the instructions	100 ms	1	vTaskDelay, LCD and UART mutexes
audio_task	Starts the codec and the I2S stream, then ends	once	1	vTaskDelay
Note timer (ticker)	Steps through the notes of the song	per note	3 (timer task)	timer expiry
Confirm timer (timeout)	Cancels a pending song change after 5 s	5 s one-shot	3 (timer task)	timer expiry

All application threads have priority 1 and give up the processor after each update with vTaskDelay, so they take turns in a cooperative style. The software timers run in the FreeRTOS timer service task, which has priority 3 so that note changes are timely. The audio DMA interrupt has a priority number of 6, which is below the FreeRTOS system-call limit, because it does not call any FreeRTOS function.

Shared resources are protected as follows. lcd_mutex makes LCD access exclusive between the LCD thread and the volume thread. uart_mutex protects USART1 output from several tasks. The player state (ui_state, pending_song, player_playing) is shared through volatile variables that are written by one task and read by another.

The handout asks for a ticker and a timeout. FreeRTOS has no such objects, so both are software timers. The note timer is a one-shot timer that is re-armed with the length of each note, because the notes have different lengths. The confirm timer is a one-shot timer started when a song is chosen.

4. Implementation

4.1 Clock

The system clock is 168 MHz, generated from the 8 MHz crystal (divided by 8, multiplied by 336, divided by 2). If the crystal does not start, the code falls back to the internal oscillator. A second PLL (PLLI2S) runs at 135.5 MHz and gives the I2S a sample rate of 44.108 kHz.

4.2 Audio Path

The codec is configured over I2C2 at address 0x10: it is muted while the registers are written, set as an I2S slave with a 256 x fs master clock and 16-bit I2S data, the DAC and outputs are enabled, the state machine is restarted, and then it is unmuted. The STM32 sends audio on I2S3 as master with DMA in circular mode. The buffer holds two halves of 256 stereo frames. Each time the DMA finishes a half, its callback refills it, so a new buffer is needed about every 5.8 ms.

4.3 Note Playback

The tone generator uses a 32-bit phase accumulator and produces a triangle wave. The phase step is the note frequency multiplied by 2 to the power of 32 and divided by the sample rate. The values in song_def.h are periods in milliseconds, so the frequency is 1000 divided by the value (for example 3.822 ms is 261.6 Hz); a value of 0 is a rest. The length of a note is its beat value multiplied by the song tempo and a tempo scale of 6.0, in seconds. The last 15 percent of each note (at least 8 ms) is silent, so repeated notes can be heard separately.

4.4 LCD

The ST7789 is driven through the FSMC (bank 1, NE3) in 8-bit mode. Address line A18 acts as the data/command select. A custom driver sets the window and writes RGB565 pixels. The text uses an 8x16 bitmap font that was generated from DejaVu Sans Mono Bold, enlarged by integer scaling. The screen shows the title, the song number with its binary code, the song name on two lines, the player state, the volume and a help line.

4.4.1 Song Selection and Buttons

polling_buttons reads the keys every 20 ms and counts a key as pressed after two equal readings. While button 1 (UP) is down, every code key (LEFT = 4, DOWN = 2, RIGHT = 1) that is seen is added to the code, so the order of pressing does not matter. The choice is made when UP is released. In the IDLE state this starts the confirmation window; in CONFIRM it confirms the pending song. The USER_BUTTON stops or plays the song on a tap, and a hold of 1.5 seconds makes the tempo faster in steps of 15 percent. A USER_BUTTON that reads pressed from start-up (for example because of a wiring fault) is ignored, and the LCD shows a warning.

4.5 Volume

The potentiometer on PF6 is read with ADC3 (12-bit, 480-cycle sampling, average of 8 readings). The value is turned into a percentage and written to the codec's digital volume registers: 0 percent is mute, and 1 to 100 percent covers -60 dB to 0 dB. A change must be at least 2 percent before it is acted on, to avoid jitter. The volume is shown on the LCD and printed on UART.

4.6 UART and Sleep Mode

USART1 runs at 115200 baud through the ST-Link virtual COM port. At start-up it prints the list of songs with their binary codes and the instructions. Events such as a selection, a confirmation or a time-out are printed afterwards. For sleep mode, the idle hook executes the WFI instruction, so the processor sleeps whenever no task is ready, and debug access during sleep is enabled so programming still works.

4.7 Song Data

The instructor's `song.h` and `song_def.h` define the songs as C++ objects with `std::string` names. `songs.cpp` is the only file that includes them. It exposes the number of notes, the tempo, the names, and each note and beat through plain C functions, and uses the smaller of the declared length and the real array sizes so a wrong length cannot read past an array.

5. Verification and Testing

5.1 Test Setup

The firmware was built with PlatformIO and uploaded through the ST-Link. Behavior was observed on the LCD, the LEDs, the speaker and the UART console (115200 baud).

5.2 Functional Tests

Test ID	Stimulus	Expected	Observed	Result
FT-01	Power on	LCD shows MP3 PLAYER; UART prints the song list and instructions	It met the expected output.	Passed
FT-02	Hold DOWN + RIGHT, tap UP	LCD shows CHANGE TO 4 011 with a countdown; green LED on	It shows song 4 Turkish March and lets you confirm for 5sec.	Passed
FT-03	Tap UP again within 5 s	Turkish March plays from the start; blue LED on; name on LCD	Fur Elise plays on the start since it is the first song listed.	Passed
FT-04	Choose a song, wait 5 s	Choice is cancelled; current song carries on	It met the expected output.	Passed
FT-05	Tap UP with no code key held	Code 000 selects the first song (Fur Elise)	It met the expected output.	Passed
FT-06	Press UP before the code keys	Same code is selected (key order does not matter)	It met the expected output.	Passed

Test ID	Stimulus	Expected	Observed	Result
FT-07	Tap USER_BUTTON while playing	Song pauses; red LED on	It met the expected output.	Passed
FT-08	Tap USER_BUTTON again	Song resumes; yellow LED on	It met the expected output.	Passed
FT-09	Turn the potentiometer	Volume changes; LCD and UART show the percentage	It met the expected output.	Passed
FT-10	Let a song finish	Player stops; red LED on; it can be replayed	The song replays when USER_BUTTON is pressed.	Passed
FT-11	Hold USER_BUTTON for 1.5 s	Tempo number on the LCD becomes smaller; notes are faster	It met the expected output.	Passed

6. Engineering Discussion

6.1 Problems and Solutions

Problem	Cause	Solution
The board was not detected and upload failed	A charge-only USB cable, and no driver for the ST-Link Debug device	Used a data cable on the ST-Link port and installed the ST-Link driver
FreeRTOS would not build	The kernel files were not in their own library folder, so PlatformIO compiled every port, and the FPU flags were not given to the linker	Placed the kernel in `lib/FreeRTOS-Kernel` with a `library.json` source filter and added the FPU flags with a small build script
The audio was only static	The audio clock came from the less precise internal oscillator, the sample-rate dividers were off, and the codec state machine had not been restarted	Switched to the 8 MHz crystal, corrected the I2S clock to 44.1 kHz and restarted the codec state machine
An LED flickered or stayed dark	A resistor was not seated firmly in the breadboard	Reseated the resistor and checked the contacts

Problem	Cause	Solution
The LCD and LEDs dimmed when the external button was pressed	The button was wired from 3.3 V to GND, which is a short circuit	Wired it between PA0 and GND only, using the internal pull-up
Only the last songs could be selected	The button input was stuck low, so it always added to the song code	Fixed the wiring and made the code ignore an input that is stuck at start-up
The songs sounded too fast or too slow	The original timing code of the handout was not available	Defined the note length as beat x tempo x scale and tuned the scale by ear to 6.0, with a way to make it faster

6.2 Design Decisions

The screen is drawn only through a mutex because two threads draw on it: one for the song information and one for the volume. A single audio buffer that is refilled by the DMA callbacks was chosen so that each note can have a different pitch, which a fixed buffer cannot do. The three code buttons and the select button use the four on-board keys, and the fifth required input (USER_BUTTON) is a breadboard button, because the RT-Spark has only four usable keys.

6.3 Differences from the Handout

- The program is `main.c` in a PlatformIO project (STM32 HAL and FreeRTOS), not a `main.cpp` made with CubeMX or CubeIDE. The mbed `main.cpp` was used only as a reference.
- Three single LEDs are used instead of an RGB LED.
- Only the first 8 of the 10 songs are selectable.
- The tempo scale was tuned by ear, because the original timing code was not available.
- The USER_BUTTON also changes the tempo when held, which is an addition.

6.4 Limitations

The sound is a simple triangle wave and not an instrument. The tempo of different songs may not all sound right with one scale. Most of the system was checked on the board by listening and observing, and there are no automated tests for the player logic.

7. Conclusion

This laboratory activity produced a working personal MP3 player on the RT-Spark. It plays songs through the audio codec, displays the song and its state on the LCD and LEDs, selects songs with a binary code and a 5-second confirmation, stops and plays from a separate button, controls volume with a potentiometer and prints its instructions over UART. The work gave practical experience with FreeRTOS tasks, software timers and mutexes, interrupt-driven DMA audio, driving an LCD directly over the FSMC, debouncing inputs and designing a small state machine.

Future improvements could include an RGB LED, a better sound than a triangle wave, a tempo for each song, a separate button for each function, automated tests for the selection logic and saving the volume and tempo.

Requirements Traceability Matrix

Requirement	Implementation	Verification
FR-01 Choose a song with a binary code on buttons 2-4 and select with button 1	polling_buttons, songs.cpp	FT-02, FT-05, FT-06
FR-02 Confirm within 5 seconds, otherwise carry on	Confirm timer (timeout), on_up_tap	FT-03, FT-04
FR-03 Show the song name on the LCD and a confirmation message	update_lcd_leds_thread	FT-02, FT-03
FR-04 Stop and replay a song	USER_BUTTON in polling_buttons	FT-07, FT-08, FT-10
FR-05 LEDs show playing, paused and changing	update_lcd_leds_thread	FT-02, FT-03, FT-07
FR-06 Adjust the volume	adjust_volume	FT-09
FR-07 Send the instructions over UART	print_banner	FT-01
FR-08 Play the music with the ticker and the timeout	Note timer, confirm timer	FT-03, FT-04, FT-10
FR-09 Three threads with infinite loops	Three threads in main.c	Code inspection
FR-10 Mutex for the LCD	lcd_mutex	Code inspection
FR-11 Clear the LCD, start the threads and use sleep mode	main, idle hook (WFI)	FT-01

References

1. BCA182 Embedded Systems Programming - Laboratory Activity 2: Personal MP3 Player, Mindanao State University - Iligan Institute of Technology, February 2026.
2. FreeRTOS Kernel documentation.
3. STM32Cube HAL and STM32F4 reference manual, STMicroelectronics.
4. ES8388 audio codec datasheet, Everest Semiconductor.
5. RT-Thread ES8388 audio driver (Apache-2.0) and RT-Spark board documentation, RT-Thread.
6. ST7789 display controller datasheet.
7. PlatformIO documentation.